Saraswata Chatterjee (CSE IoT 2nd Yr) (12024052019003)

Swing

Student Information System

Section A: GUI Architecture Overview

Overall Structure: The application follows a **single-window desktop GUI architecture** built using Java Swing. The design separates:

- **User Interface (UI components)**
- **Event handling logic**
- **Data storage and file persistence**

The program runs on the **Event Dispatch Thread (EDT)** using: `SwingUtilities.invokeLater(() -> new StudentInfoApp());`

Main Window

- Implemented using JFrame
- Title: *Student Information System*
- Layout: BorderLayout

Input Components

- JTextField → Student Name input
- JComboBox → Course selection
- JButton → Save, Clear
- JTextField → Search field
- JButton → Search Name

Display Components

- JTable → Displays student records
- DefaultTableModel → Stores table data
- JScrollPane → Enables scrolling
- JMenuBar → File & Help menu

Data Storage Structure

- **Runtime Storage**: DefaultTableModel
- **Persistent Storage**: students.txt
- Format: Name,Course

Control Flow

1. User enters data.
2. Clicks **Save**.
3. ActionListener triggers handleSave().
4. Data is validated.
5. Added to tableModel.
6. Saved to file.
7. Table updates automatically.

UI → Event → Logic → Model → File → UI refresh

Section B: Swing Component Selection

JFrame

- **Purpose:** Main application window.
- **Why appropriate:** Top-level container for Swing apps.
- **Limitation:** Only one root window; complex apps need multiple frames or dialogs.

JTextField

- **Purpose:** Input student name and search text.
- **Why appropriate:** Designed for single-line input.

- **Limitation:** Does not support multi-line input.

JComboBox

- **Purpose:** Select course.
- **Why appropriate:** Restricts input to predefined options.
- **Limitation:** Limited scalability if options become very large.

JButton

- **Purpose:** Trigger Save, Clear, Search actions.
- **Why appropriate:** Standard event-trigger component.
- **Limitation:** Requires event listener for functionality.

JTable

- **Purpose:** Display structured student data.
- **Why appropriate:** Tabular data display with sorting support.
- **Limitation:** Complex customization requires additional configuration.

<u>Section C: Event Handling and Listener Logic</u>

Events Generated

- Save button click
- Clear button click
- Search button click
- Menu item click (Exit, About)

ActionListner Processing

Lambda expressions are used: saveButton.addActionListener(e -> handleSave());

When triggered:

- Data is retrieved
- Validation occurs
- Model updates
- File saved

Data Transfer Process

1. nameField.getText()
2. courseBox.getSelectedItem()
3. Data added via tableModel.addRow()
4. Persisted via saveToFile()
5. Display updated automatically

<u>Section D: Validation and Error Handling</u>

```
if (name.isEmpty()) {
   JOptionPane.showMessageDialog(frame, "Name is required!", "Error",
JOptionPane.ERROR_MESSAGE);
   return;
}
```

On Invalid Input

- Error dialog appears.
- Data is NOT added.
- File is NOT modified.

Why Validation is Necessary

- Prevents empty records
- Maintains data consistency
- Avoids corrupt file entries
- Improves user experience

Section E: Layout Management Reasoning

Layout Managers Used

- BorderLayout → Main frame
- GridBagLayout → Input panel
- FlowLayout → Search panel & button panel

Why These Layouts?

BorderLayout

- Separates top controls and center table cleanly.

GridBagLayout

- Flexible grid positioning for labels and fields.
- Allows resizing adaptability.

FlowLayout

- Simple left-aligned search components.

Resizing Behavior

- JTable expands with window.
- Top panels remain structured.

This ensures responsive UI without distortion.

Section F: Data Handling and Persistence

Data Storage During Runtime

Stored in DefaultTableModel.

File Persistence

File: students.txt

Format: John,Math
Sara,Science

Save Process

- Loops through table rows
- Writes comma-separated values

Load Process

- Reads line-by-line
- Splits using .split(",")
- Adds rows to table model

UI and File Consistency

- After every Save → File overwritten with current table state.
- Prevents mismatch between UI and file.

Section G: What-If Design Analysis

1. What problems arise if all logic is written inside event handlers?

- Code becomes messy and unmaintainable.
- Difficult to debug.
- Hard to reuse logic.
- Violates separation of concerns principle.

Your project improves this by using separate methods like:

- handleSave()
- saveToFile()
- loadFromFile()

4. What happens if long tasks run on the Event Dispatch Thread?

- UI freezes.
- Buttons stop responding.
- Window becomes unresponsive.

Swing is single-threaded; heavy tasks should use SwingWorker.

Section H: Individual Extension Explanation

Implemented Extension: Search Functionality with TableRowSorter

I used: TableRowSorter<DefaultTableModel>

RowFilter.regexFilter("(?i)" + text)

Impact on UI Flow

- User enters search text.
- Table dynamically filters.
- Case-insensitive search.
- Improves usability significantly.

This makes the application interactive and scalable.

Section I: Debugging and Learning Reflection

Issue Encountered

If file did not exist initially, application could fail while loading.

Diagnosis

Used: if (!file.exists()) return;

Learning Improvement

- Understood importance of file existence checks.
- Learned difference between runtime and persistent storage.
- Improved understanding of MVC-like separation in Swing.

Section J: AI Usage Disclosure

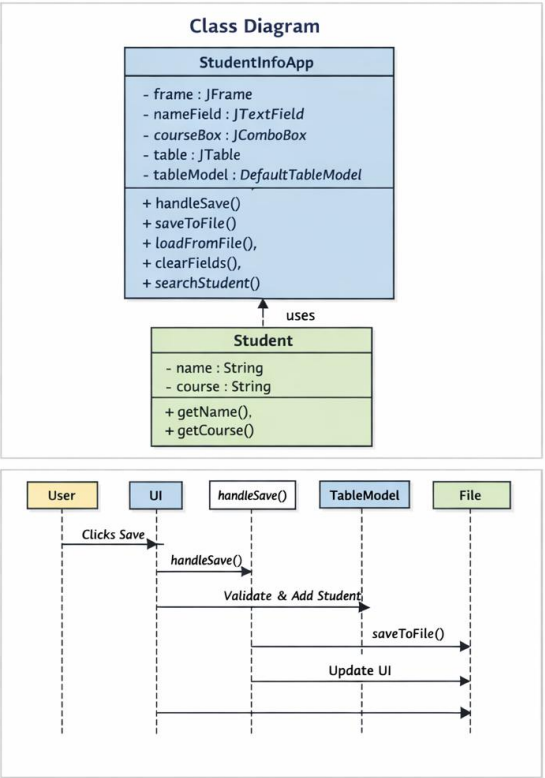
AI assistance was used for:

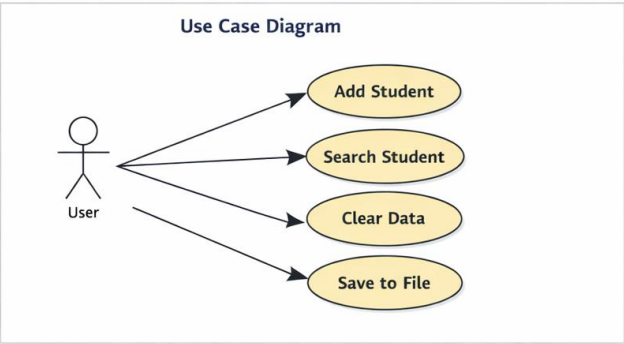
- Structuring the project report.

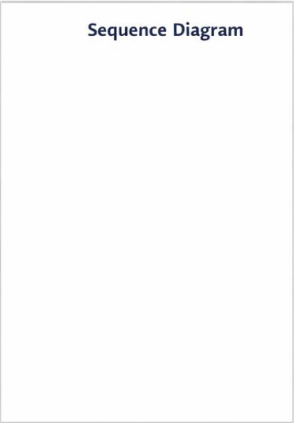
Usability was validated by checking:

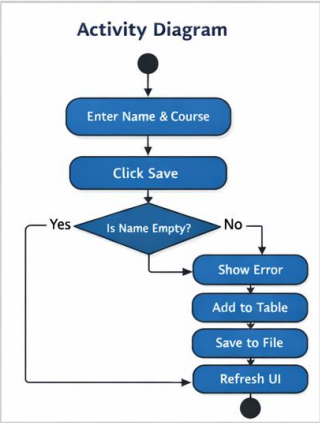
- Proper error dialogues

UML Diagram

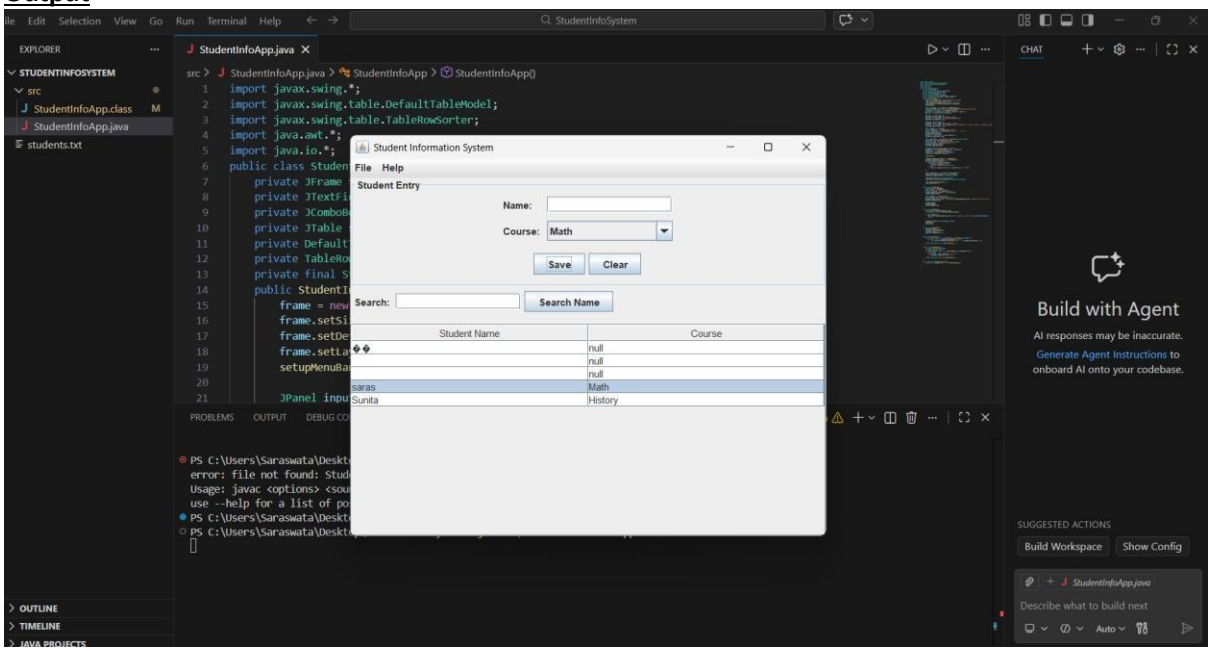








Output


StudentInfoApp.java
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import javax.swing.table.TableRowSorter;
import java.awt.*;
import java.io.*;
Student Information System
File Help
Student Entry
Name:
Course: Math
Save
Clear
Search:
Search Name
Student Name
Course
null
null
null
saras
Math
Sunita
History
Build with Agent
AI responses may be inaccurate.
Generate Agent Instructions to onboard AI onto your codebase.
SUGGESTED ACTIONS
Build Workspace
Show Config
Describe what to build next